

Y a-t-il une douane avant S . foodLog ?

Force Tracker — 11/09/2026. Cartographie demandee par Michel **avant** de decider d'une refonte : « existe-t-il aujourd'hui un *UNIQUE point final* qui normalise et valide toute entree nutritionnelle avant ecriture, quelle que soit son origine ? » **Trace dans le depot, pas de memoire.** Les comptes de ce document sont recalculés a chaque generation depuis app . js : si le code change, le document change ou refuse de se produire.

LA REPONSE : C — NON

Trois fonctions ecrivent dans S . foodLog, chacune avec ses propres regles, et **deux des trois ne croisent aucun controle**. Il existe bien un morceau commun aux trois (`_provFood`), et **c'est exactement le piege** : il recopie de la *provenance*, il ne valide rien. Aucune verification de quantite, d'unite, de pour-100 g, de calories ou de macros n'y passe.

Ce n'est pas un oubli, c'est une forme. Le point commun est une *liste blanche de recopie*, pas une douane : chaque nouveau champ doit etre ajoute a la main dans chaque porte, et l'oubli est silencieux.

1. Les 3 sorties

Mesure : `grep 'S . foodLog . push( ' sur tout le depot rend 3 sites`, tous dans app . js.

fonction qui ecrit	ligne	controles traverses avant l'ecriture
<code>addFoodEntry()</code>	4024	nom non vide · au moins une valeur · geste quantite (<code>_bcQtyPose</code>)
<code>quickAddFood()</code>	2796	aucun
<code>rejouerRepas()</code>	2138	le <i>moment du repas</i> seulement, valide contre <code>FOOD_MEALS</code> — rien sur les valeurs

Plus deux chemins qui remplacent le tableau entier sans rien regarder : la restauration cloud (`setup . js`) et le chargement local (`state . js`). Ils sont hors du sujet d'une douane de saisie, mais ils existent et meritent d'etre nommes : aucune regle de qualite ne survit a un remplacement en bloc.

La porte sans aucun controle, extraite du depot. Commentaires retires ; les messages affiches sont abreges en ' . . . ' (la police du PDF ne rend pas les emoji).

```
function quickAddFood(i){
  try{ _afOublierAliment(); }catch(e){}
  const it=_afQuickItems[i]; if(!it)return;
  if(!S.foodLog)S.foodLog=[];
  const _vals={kcal:it.kcal||0,prot:it.prot||0,carbs:it.carbs||0,fat:it.fat||0};
  const _qOk=(+it.q>0 && (!it.u||it.u==='g'||it.u==='portion'));
  _afSetSrc({saisie:'liste', origine:'reprise',
    q:_qOk ? +it.q : null, u:_qOk ? (it.u||'g') : null,
    per100:it.per100||null, sourceId:it.sourceId||null, etat:it.etat||null,
    portionLabel:it.portionLabel||null, portionWeightG:+it.portionWeightG>0?+it.portionWeightG:null})
  » ;
  S.foodLog.push(Object.assign({date:_journalJourActif(),meal:_afMeal,name:(it.name||'').slice(0,80),ts:Date.n
  » ow(),_vals,_provFood(_vals)});
  _afSetSrc(null);
  _unhideFood(it.name);
  persist(); if(typeof _cloudSyncDebounce==='function')_cloudSyncDebounce();
```

2. Les huit portes d'entree

Chacune fabrique elle-meme l'objet pour-100 g (`_bcNutr`). Mesure : **8** constructeurs distincts dans le depot.

porte	fabrique	transforme	hub ?	controle
scan code-barres	<code>_lookupBarcode</code>	<code>_per100d1</code>	oui	formulaire (avis)
code-barres tape	<code>_bcManuel -> idem</code>	<code>_per100d1 + codeDouteux</code>	oui	formulaire (avis)
recherche Open Food Facts	<code>_afSuggPrendreOff</code>	<code>_per100d1</code>	oui	formulaire (avis)
recherche CIQUAL	<code>_afSuggPrendreCiqua l</code>	<code>_per100d1</code>	oui	formulaire (avis)
marques	<code>_afSuggPrendreMarqu e</code>	<code>_per100d1</code>	oui	formulaire (avis)
photo d'etiquette	<code>onFoodLabelFile</code>	<code>_per100d1</code>	non	formulaire (avis)
estimation IA / etiquette recopiee	<code>_calAppliquer</code>	<code>_per100d1</code>	oui	bloquant
Mes aliments / deja note	<code>quickFillFood · _afSu ggPrendreLocale</code>	valeur brute	non	aucun si ajout direct
saisie manuelle	les 4 champs	<code>parseInt</code>	—	formulaire (avis)

Le hub `_offRemplirFormulaire` couvre **5 portes sur 8**. Les trois autres en recopient une version raccourcie, chacune avec ses propres omissions.

3. Les regles dupliquees ou divergentes

3.1 — La meme regle physique, appliquee a deux forces differentes

`_masseImpossibleVals` et `_kcalImpossibleVals` sont des fonctions **pures**, a proprietaire unique : sur ce point l'architecture est saine. Mais elles ne sont pas *appliquees* de la meme facon.

Porte IA / etiquette recopiee — elles BLOQUENT : `return`; sec, la valeur n'entre pas. Messages abreges.

```
const imp=_masseImpossibleVals(100, prot, carbs, fat);
if(imp){ dire('...' + imp.somme + '...'); return; }
const kimp=_kcalImpossibleVals(100, kcal, prot, carbs, fat);
if(kimp){ dire('...' + kimp.kcal + ' kcal pour 100 g : impossible avec ces macros. Elles valent '+kimp.theo+'
» kcal, et les '+kimp.libre+' g restants ne peuvent pas dépasser '+kimp.plafond+'...'); return; }
```

Formulaire — elles AFFICHENT, et rien de plus : `montrer()`; `return`; . Messages abreges.

```
const masse=_masseImpossible(pfx);
if(masse){
  el.innerHTML='...' + masse.somme + ' g</b> de protéines, glucides et lipides ne tiennent pas dans '
  + '<b>' + masse.q + ' g</b>. Un aliment ne peut pas contenir plus de matière qu'il ne pèse.'
  + '<div style="color:var(--t3);margin-top:6px;">Soit la quantité, soit une des trois valeurs '
  + '...';
```

LA CONSEQUENCE, EN UNE PHRASE

La **meme** erreur est **refusee** par une porte et **enregistree** par la porte d'a cote. Et `addFoodEntry` n'appelle jamais le controle de coherence avant d'ecrire : l'avertissement a pu s'afficher pendant la frappe, il ne bloque pas la sortie.

3.2 — 8 constructeurs du pour-100 g, dont 2 sans normalisation

Mesure a l'execution : **6** passent par `_per100d1` (lignes 1667, 1806, 2901, 3470, 3497, 3943) et **2** prennent la valeur brute (lignes 2694, 3853). Les deux bruts sont les chemins de *reprise* — defendable, puisque la valeur a deja ete arrondie une fois ; mais la regle tient alors a l'**historique de la donnee**, pas au code.

3.3 — Le hub recopie en deux variantes

onFoodLabelFile refait a la main les lignes du hub **en omettant** : etat : ' tel-que-vendu ' · sourceId · _bcPaquetG/_bcProposerPaquet · _afNoteEtat · _bcProposerDerniere(0) · la carte sante. quickFillFood et _afSuggPrendreLocale en omettent une autre liste. **Ce ne sont pas des bugs isolés : c'est la meme fonction ecrite trois fois, qui derive.**

3.4 — Le garde-fou quantite tient a un ENDROIT, pas a une regle

Il vit dans addFoodEntry. Les deux autres sorties ne le croisent pas. C'est *defendable* (elles recopient une quantite deja stockee) mais la protection vient de la **position du code**, pas d'un controle. Le jour ou une quatrieme sortie apparait, rien ne la protege et rien ne le signale.

3.5 — _provFood est une liste blanche opt-in

C'est le seul point commun aux 3 sorties. Il ne valide rien : il recopie, champ par champ, ce que la porte a pose dans _afSrc. **Un champ non liste disparaît sans erreur et sans test rouge.**

L'objet de depart : tout ce qui n'est pas recopie ensuite reste a null.

```
function _provFood(vals){
  const p={v:FOOD_LOG_V, saisie:'manuel', origine:'utilisateur',
    q:null, u:null, etat:null, sourceId:null, per100:null, modifie:false};
  ...
}
```

LE FICHIER COMPTE LUI-MEME SES OUBLIS — QUATRE FOIS AU MEME ENDROIT

Les commentaires de cette fonction, ecrits en majuscules par les versions successives, recensent quatre oublis du **meme** type : codeDouteux · q/u · portionLabel/portionWeightG · doute/kcalDerivee.

C'est la signature structurelle d'un recopieur, pas d'un validateur. Le troisieme commentaire le dit en toutes lettres : « j'ai ecrit la regle juste au-dessus de l'endroit ou je venais de l'enfreindre ». Quand l'erreur se repete au meme endroit malgre un avertissement en majuscules, ce n'est plus de l'inattention : c'est la forme du code qui la produit.

3.6 — Latent, mais reel : la valeur controlee n'est pas la valeur ecrite

addFoodEntry ecrit avec parseInt, tout le reste lit avec numFR (virgule et decimale). **Sans effet aujourd'hui** — les totaux affiches sont deja entiers — donc ce n'est pas un bug a corriger dans l'urgence. Mais c'est la forme classique : *on verifie une valeur, on en enregistre une autre.*

4. Ce que ca veut dire pour la suite

La question de Michel derriere la question : « *le vrai probleme de Nutrition vient-il du fait qu'on corrige les portes une par une au lieu d'avoir une seule douane finale ?* »

Oui, et c'est mesurable. Sur les dernieres versions nutrition, le motif est toujours le meme : *la porte jumelle avait le meme default.* La regle **R8** est citee huit fois dans le journal, et une famille de bugs a ete ecrite expres pour ca (BUGS.md §59) : « *le code corrige est juste, les temoins verts le meritent, la mutation mord — et la moitie du bug est encore la* ».

LA RESERVE QUI CHANGE LE CHANTIER

La douane ne peut pas vivre dans _provFood. Il ne voit que les quatre totaux et l'objet _afSrc ; il ne voit ni la quantite effective, ni le contexte de la porte, ni le pour-100 g reel.

Une douane utile se place **entre les 3 push et S. foodLog** — c'est-a-dire un **quatrieme point qui n'existe pas encore**. Ce n'est donc pas un deplacement de code : c'est une piece a ajouter, avec ses temoins.

Ce que la douane devrait faire (a discuter, rien n'est decide)

role	aujourd'hui	dans une douane
normaliser le pour-100 g	6 portes sur 8	une fois, pour tout le monde
regles physiques (masse, plafond kcal)	bloquantes sur 1 porte, en avis sur les autres	une seule force, decidee une fois
quantite et unite	recopiees par une liste blanche opt-in	champ obligatoire ou absence declaree
provenance	_provFood (recopie, 4 oublis recenses)	liste fermee : un champ inconnu leve, il ne disparaît pas
coherence kcal / macros	affichee dans le formulaire, jamais bloquante	un etat nomme ecrit sur la ligne

CE QUE CE DOCUMENT N'EST PAS

Aucun correctif n'a été écrit, aucune ligne de production n'a changé. Michel a demandé une cartographie et une conclusion **avant** de décider d'une refonte : c'est tout ce que contient ce document.

Et une honnêteté sur la méthode : les comptes ci-dessus sont recalculés à chaque génération, mais **la lecture de ce qu'ils signifient reste la mienne**. Le générateur peut prouver qu'il y a 3 sorties ; il ne peut pas prouver que deux d'entre elles *devraient* croiser un contrôle.